

MIA's RPG System Design & Programming Documentation

Developer: Antonius Torode

Version – Version 0.001
Latest update: September 24, 2026

© 2025 Antonius Torode
All rights reserved.

This work may be distributed and/or modified under the conditions of Antonius' General Purpose License (AGPL).

The Original Maintainer of this work is: Antonius Torode.

The Current Maintainer of this work is: Antonius Torode.

This document provides documentation for an RPG system designed within Multiple Integrated Applications (MIA), a program created for personal use. Herein, "MIA" refers to "Multiple Integrated Applications." This acronym is used exclusively for this program and was coined by its creator. The RPG system within MIA is built to support flexible, expandable gameplay mechanics, allowing continuous addition of features and adaptability for various gaming scenarios and purposes.

This document is continuously under development.
Most Current Revision Date:

September 24, 2026

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Power Progression	1
1.3	Expansions	1
1.4	Crafting System	2
1.5	Character Progression and Identity	2
1.6	Gear and Itemization	2
1.7	Stat System and Character Customization	3
1.8	Pace and the Journey	3
2	Design	4
2.1	Goals	4
2.2	Configuration	6
2.2.1	Vision	6
2.2.2	Configuration Design	8

This page intentionally left blank.
(Yes, this is a contradiction)

Chapter 1

Introduction

1.1 Introduction

This RPG system is an attempt to create a comprehensive RPG system to be used by an MMORPG. This system is inspired by the classic World Of Warcraft (WoW) game. It attempts to take elements which made the original WoW game great, and incorporate them. It also attempts to take elements of the newer versions of the game which are bad, and remedy them in a way that preserves the essence of what made the original versions of WoW good. This chapter will outline many of the features which may or may not be directly developed in this RPG system, but are key in mind during development.

1.2 Power Progression

One of the fundamental goals of this RPG system is to ensure that character progression feels meaningful at every stage of the game. In contrast to modern World of Warcraft (WoW), where leveling often results in a temporary weakening of the character relative to the world until max level is reached and gear progression begins, this system adopts a more linear and intuitive approach.

Inspired by classic WoW, every improvement, whether it be a new ability, a talent point, a stat increase, or a piece of gear, represents a true and permanent gain in power. The world scales around the player in a fixed manner, and character growth is never undone or diminished by artificial re-scaling or squish mechanics. This means that as players invest time into their characters, they will feel steadily more capable and powerful, with no regression or power dilution.

This model emphasizes consistent and rewarding progression, preserving the classic RPG ideal that advancement, whether through experience, training, or equipment, should always be a step forward.

1.3 Expansions

A core design philosophy of this system is that expansions should truly expand existing game systems, not replace or invalidate them. In many modern MMORPGs, and particularly in the evolution of World of Warcraft, expansions frequently override previous mechanics, render earlier content obsolete, and disrupt continuity in gameplay.

This RPG system is designed to avoid that failure. Expansions will be additive in nature, introducing new content, systems, and progression paths while preserving and reinforcing what already exists. All previously developed mechanics, zones, professions, gear, and abilities will remain accessible and relevant.

New content introduced through expansions will build upon the established foundation. Rather than forcing players to abandon past achievements or systems, expansions will increase the breadth and depth of the game world. The goal is to maintain a persistent, interconnected world where every stage of progression remains meaningful, regardless of when it was introduced.

1.4 Crafting System

Crafting is a foundational element of this RPG system, designed to be meaningful, rewarding, and relevant throughout a character's entire journey. Unlike in many modern MMORPGs, where crafting is often relegated to endgame utility and reduced to a passive grind, this system treats crafting as an integral path of character development and player interaction.

Professions and tools will play an active role in gameplay. Every profession will provide distinct value, with deliberate overlap to ensure no profession becomes obsolete or isolated. Players will be able to specialize in meaningful ways, and crafting will support both combat and non-combat roles. Whether creating gear, consumables, materials, or tools, crafted items will remain viable and useful across all levels of play.

As characters grow in power, so too will their crafting abilities. Crafting will not be an afterthought or a side activity to be rushed at max level, but a progression path that naturally evolves alongside combat and exploration. This system encourages economic interdependence, where players benefit from trading, collaboration, and the diverse outputs of various professions. Crafting is not just a system, it is a core part of the game's identity, enabling players to shape the world, their characters, and their community through meaningful, consistent engagement.

1.5 Character Progression and Identity

In this system, character growth is driven by player agency and in-world interaction, not by automated progression or rigid class templates. Unlike modern World of Warcraft, where spells and abilities are granted automatically with level-ups and class choice is fixed at character creation, this system offers a dynamic model of progression that reflects the player's actions and choices.

There are no predefined classes. Instead, characters develop identities through the paths they follow. The spells, abilities, and skills a character learns are determined by where they go, who they interact with, what they study, and how they spend their time. Progression is earned, not given. The more a character practices a skill, the more proficient they become. The more they explore a discipline, the more it unfolds for them.

This design gives players full control over how their characters evolve. A single character might grow into a warrior through constant combat, a spellcaster through arcane study, or something in between based on a unique combination of choices. Identities are shaped organically, not selected from a list.

This approach ensures that every character reflects the time, effort, and decisions of the player. Progression feels meaningful because it is the direct result of engagement with the world, not the product of a scripted leveling sequence.

1.6 Gear and Itemization

Gear in this system is a reflection of a character's journey, effort, and choices, not something handed out freely. Modern World of Warcraft has diluted the meaning of gear progression, making high-end items easily accessible and stripping rarity of its impact. This system returns to a model where gear is meaningful, and where rarity is not just a label, but an indicator of value, effort, and identity.

Items will exist across familiar rarity tiers, but those tiers will be tied directly to difficulty, effort, and sometimes chance. Rare and powerful items will be earned through difficult encounters, persistent exploration, or unique discoveries. They will not be guaranteed, and not every player will possess them. This ensures that standout gear remains exactly that - standout.

The average player's gear will reflect their chosen activities and investment. A player who explores dangerous ruins, masters a crafting discipline, or builds a network of allies will be rewarded with different gear outcomes than one who focuses solely on combat or trade. This makes gear a narrative and mechanical extension of the character, not just a stat stick.

With this approach, seeing a player wearing a particular item will tell a story. It restores a sense of achievement, recognition, and individuality that modern systems have largely discarded in the name of accessibility.

1.7 Stat System and Character Customization

The stat system in this RPG is a central element of player-driven character development. Unlike modern World of Warcraft, where stats are tied almost entirely to item level and increase passively with gear upgrades, this system makes stats an intentional and impactful part of character identity.

Players will select and develop their stats as they see fit. Every stat influences gameplay in a specific and tangible way, whether it's casting speed, damage output, movement precision, or defensive efficiency. These choices affect how a character functions in combat, exploration, and other systems. Growth is directed by the player, not dictated by external scaling.

There are no predefined builds or limitations on how stats must be arranged. Characters can develop along conventional paths or explore creative combinations to support unique playstyles. This freedom allows for a wide range of viable builds, each shaped by the player's decisions and goals.

In this system, stats are not trivial background numbers. They are a reflection of how the character has grown, what the player values, and how they choose to engage with the game world. This makes character progression both meaningful and personal.

1.8 Pace and the Journey

This RPG system is built around the idea that the journey is not a barrier to the game, it is the game. Unlike modern World of Warcraft, where the focus is on reaching max level as quickly as possible so that the "real" content can begin, this system places value on every step of a character's progression.

The pace is deliberately designed to be slow and immersive. It is intended to give players time to explore, interact, learn, and grow. Whether a player is casual or dedicated, the experience offers meaningful engagement throughout. Leveling is not something to rush through or skip; it provides purpose, discovery, and connection.

There is always something to do and someone to meet. Progression systems, crafting, exploration, and community interaction are all tightly woven into the leveling process. This encourages natural cooperation and social bonds, making the world feel alive and connected.

By avoiding artificial urgency and grind-focused design, the game fosters a space where players can enjoy the world for what it is, and not just as a means to an end. The result is a game that rewards attention, effort, and presence, not just speed.

Chapter 2

Design

This chapter will cover design ideas for the MIA RPG system. This chapter will be considered a living document which may change very often. Ideas, the purpose for the ideas, and different solutions or considerations for design will be discussed.

2.1 Goals

The following goals define the core vision for the MIA RPG system, ensuring it is modular, scalable, and simple to extend for MMO development and game creation tools. Each goal includes its purpose and design considerations to guide implementation.

1. Fully Configurable Framework

- **Purpose:** Enable dynamic addition or removal of game elements (e.g., vitals, currencies, items, abilities) via configuration files, minimizing code changes and supporting tools like a game editor similar to Warcraft 3's.
- **Considerations/Solutions:**
 - Use JSON/YAML files (e.g., `vitals.json`, `items.json`) to define all game elements, including properties like name, type, and behavior.
 - Implement registries (e.g., `VitalsRegistry`, `CurrencyRegistry`) to load and manage configurations at runtime.
 - Support runtime updates via an API or scripting (e.g., Lua) for editor-driven additions.
 - Example: Add a “stamina” vital via the editor, updating `vitals.json` and reflecting in-game without recompiling.

2. Editor-Friendly Design

- **Purpose:** Facilitate a game creation editor by exposing all configurable elements (vitals, items, loot tables) in a user-friendly format, enabling non-programmers to build levels, quests, or mechanics.
- **Considerations/Solutions:**
 - Develop a GUI editor that reads/writes config files, with templates for common elements (e.g., health vital, gold currency).
 - Provide scripting support (e.g., Lua) for custom behaviors, accessible via editor UI.
 - Include validation tools to ensure config integrity (e.g., valid ranges for vital max/min).
 - Example: Editor allows dragging a “sword” item into a creature’s loot table, updating `loot.json`.

3. Interconnected Gameplay Elements

- **Purpose:** Create a rich, MMO-like experience where elements like vitals, items, and environments interact dynamically (e.g., temperature vital affected by climate zones, loot tables yielding currencies/items).

- **Considerations/Solutions:**

- Use an Entity-Component-System (ECS) to manage entities (players, NPCs, zones) with components like vitals, inventory, or stats.
- Define relationships in config files (e.g., `climate.json` links zones to temperature effects).
- Implement event-driven interactions (e.g., `onEnterZone` adjusts temperature vital).
- Example: A “frozen tundra” zone reduces the “temperature” vital, countered by equipping a “fur cloak” item.

4. Scalable MMO Infrastructure

- **Purpose:** Support MMO-scale features like large player counts, persistent worlds, and networked data access for vitals, inventories, and quests.

- **Considerations/Solutions:**

- Design a server-client architecture with a database backend (e.g., SQLite) for persistent storage of player data.
- Use efficient data structures (e.g., `std::unordered_map`) for fast access to vitals/currencies.
- Enable networked updates via events (e.g., `onVitalChanged` syncs health across clients).
- Example: Player’s “gold” currency updates in real-time across server and clients after looting.

5. Simplicity for Expansion

- **Purpose:** Ensure new elements (e.g., vitals, items) can be added via configs or editor without code changes, maintaining simplicity for developers and modders.

- **Considerations/Solutions:**

- Centralize element management in registries with generic APIs (e.g., `addVital(name, config)`).
- Use data-binding to link configs to game systems (e.g., UI, combat) automatically.
- Support modding via external config files or scripts, validated at load time.
- Example: Adding a “morale” vital via editor updates UI and gameplay without altering codebase.

6. Comprehensive Core Mechanics

- **Purpose:** Cover all essential RPG mechanics (combat, progression, crafting, exploration) with configurable parameters to support diverse MMO features.

- **Considerations/Solutions:**

- Define core systems (e.g., combat, inventory, questing) in config files with customizable rules (e.g., damage formulas, drop rates).
- Support loot tables linking items, currencies, and probabilities (e.g., `loot.json` for creature drops).
- Include environmental effects (e.g., climate impacting vitals like temperature or thirst).
- Example: A “dragon” loot table includes “gold” currency and “dragon scale” item, with configurable drop chances.

7. Robust and User-Friendly UI

- **Purpose:** Provide a dynamic UI that reflects configurable elements and supports MMO-scale interactions (e.g., trading, party management).

- **Considerations/Solutions:**

- Use a UI config file (e.g., `ui_layout.json`) to define displays for vitals, inventories, etc.
- Implement data-binding to update UI automatically when elements change (e.g., new vital added).
- Support customizable HUDs via editor for player-specific layouts.
- Example: A “health” progress bar auto-updates when a new vital is added via `vitals.json`.

2.2 Configuration

This section details the configuration strategy for the MIA RPG system, focusing on simplicity for testing, scalability for MMO deployment, and support for a game creation editor. Configuration is designed to allow dynamic addition and removal of game elements (e.g., vitals, items, loot tables) via JSON files during development, with a compressed format for release and robust server-side validation to prevent tampering.

2.2.1 Vision

The configuration system uses JSON files in code folders to define example items, vitals, loot tables, and other objects for testing and development. These will be compressed into a single, MPQ-like format for game releases, with tools to view and edit the archive, supporting editor-driven game creation. Server-side database features will store character-specific data and save states, with consideration for including item definitions to prevent tampering. A file-based “saveToFile” and “loadFromFile” system will support player data storage during testing. These files can even be stored locally for the release game, and hashed values can be checked against the server-side saves to detect tampering and restore changed or damaged files.

The configuration system will be structured to support the following objectives, ensuring simplicity, editor compatibility, and MMO-scale robustness.

1. JSON-Based Configuration for Testing

- **Purpose:** Enable rapid iteration during development by using human-readable JSON files to define game elements, facilitating testing and editor integration.
- **Design:**

- Store JSON files in relevant folders (e.g., `data/vitals/vitals.json`, `data/items/items.json`, `data/loot/loot.json`).
- Example `vitals.json`:

```

1  [
2    {"id": "health", "name": "Health", "current": 100, "max": 100, "
      min": 0},
3    {"id": "mana", "name": "Mana", "current": 50, "max": 100, "min":
      0}
4  ]

```

- Example `items.json`:

```

1  [
2    {"id": "sword_001", "name": "Iron Sword", "type": "weapon", "
      damage": 10, "tradeable": true},
3    {"id": "potion_001", "name": "Health Potion", "type": "consumable",
      },
4    "effect": {"vital": "health", "value": 20}}
5  ]

```

- Load files into registries (e.g., `VitalsRegistry`, `ItemRegistry`) using a JSON parser (e.g., `nlohmann/json` in C++).
- Include validation for required fields (e.g., `id`, `max`) and default values for optional fields (e.g., `min=0`).

2. Compressed Format for Release

- **Purpose:** Optimize game releases by bundling JSON configurations into a single, compressed archive (similar to WoW’s MPQ) to reduce file count and improve load times.
- **Design:**
 - Use a compression library (e.g., `libzip`) to create an archive (e.g., `game_data.mia`) containing all JSON files.
 - Include a manifest file (e.g., `manifest.json`) listing contents for quick access:

```

1 {
2   "files": ["vitals.json", "items.json", "loot.json"]
3 }

```

- Develop tools (e.g., C++ or Python-based editor) to extract, view, and edit the archive, supporting the game creation editor.
- Example: Editor extracts `items.json`, allows adding a new item, and recompresses into `game_data.mia`.

3. Server-Side Database with Tamper Checks

- **Purpose:** Store character-specific data and item definitions server-side to ensure persistence and prevent tampering in MMO environments.
- **Design:**
 - Use a database (e.g., SQLite for testing, scalable DBMS for release) to store player data (e.g., vitals, inventory) and static item definitions.
 - Example `items` table:

```

1 CREATE TABLE items (
2     id VARCHAR(50) PRIMARY KEY,
3     name VARCHAR(100),
4     type VARCHAR(50),
5     properties JSON
6 );

```

- Sync `items.json` with the `items` table during release to validate client actions (e.g., item usage) against server data.
- Example: Client sends `use_item("potion_001")`, server checks `properties` (e.g., heals 20 health) and applies the effect.
- Use checksums (e.g., SHA256) on the compressed archive to detect client-side tampering.

4. File-Based Player Data Storage for Testing

- **Purpose:** Provide a simple, file-based system to save and load player data during testing, mirroring future database functionality.
- **Design:**
 - Store player state in `player.json`:

```

1 {
2   "player_id": "player_001",
3   "name": "Hero",
4   "vitals": [
5     {"id": "health", "current": 80, "max": 100, "min": 0},
6     {"id": "mana", "current": 30, "max": 100, "min": 0}
7   ],
8   "inventory": [
9     {"item_id": "sword_001", "quantity": 1},
10    {"item_id": "potion_001", "quantity": 3}
11  ],
12  "currencies": [
13    {"currency_id": "gold", "amount": 500}
14  ],
15  "position": {"x": 100, "y": 200}
16 }

```

- Implement `saveToFile` and `loadFromFile` in a `Player` class using `nlohmann/json`.
- Example:

```

1 void Player::saveToFile(const std::string& path) const;
2 bool Player::loadFromFile(const std::string& path);

```

- Handle errors with defaults (e.g., new player with 100 health) and log issues to a debug console.
- Design fields to match future database schema for seamless migration.

2.2.2 Configuration Design

The configuration system will begin with defining various objects within a JSON configuration file. Each object will contain a unique ID, name, and description which will serve as an identifier for the object itself. The remaining fields will depend on the object's purpose and function. In order to have multiple defined types within the JSON files, an object type will be declared as well, to separate the various types.

```

1 {
2   "OBJECTTYPE":
3   [
4     {
5       "id": 1,
6       "name": "object name 1",
7       "description": "object description 1",
8       // Type specific values...
9     },
10    {
11      "id": 2,
12      "name": "object name 2",
13      "description": "object description 2",
14      // Type specific values...
15    },
16    // Other values...
17  ]
18 }
```

These object types will have associated data classes which match the data fields to store each type. These types will define the main objects of the RPG system and will need continual referencing to gather their data and use their values. Since these are configurable, they are not deterministic (except for their structure) at compile time. For this reason, a registry will be designed for each type which will load in all of the configurable values defined in the JSON files and store them in a unique singleton instance. This will provide a single location to gather this information.

The various types all will potentially store in different ways. For example, a vital may store some current value, as well as some min or max, whereas a currency may just be a single value. A separate storage class for each type will need designed which defines the stored values associated with it.